



SERIES #1: CRACK MNC JAVA INTERVIEW

About Series #1: Crack MNC Java Interview

Week 1: Top OOPS Questions for Interview

- 💡 Any interview starts with OOP concepts
- Think of OOPs as the bricks when you're building a house.
- That's why it's crucial to be completely perfect with OOPs before diving into any interview!

Why OOPs is Important in Interviews

- ✓ If an interviewer asks any question related to OOPs, you must be ready to answer confidently.
- ✓ That's why we've dedicated the first week to mastering OOPs concepts.

Case 1: 🎓 Especially for College Students and Experienced Professionals

- Interviewers will focus mainly on the basics of OOPs.
- So, be strong with OOPs!
- Understand the concepts thoroughly.
- Be prepared to explain:
 - How you applied OOPs in your college project
 - How OOPs help in developing real-time applications (this is very important!).

Why First Impressions Matter

- ✨ The first impression is the best impression!
- This is where you set the tone for the entire interview.

👉 We'll provide you with:

- A list of questions.
- A sample answer pattern to help you frame your answers

Adapt this pattern to give structured, confident answers.

Pro Tip:

- 📝 Giving the best answer for OOPs questions increases your chances of clearing the interview by 75%!
- Top OOPs Topics to Cover for Java Interviews
- Before attending any interview, make sure you've mastered

About Series #1: inheritance Interview FAQ

Inheritance

- Inheritance is a one of the Object-Oriented Programming principles
- It is OOPS programming technique where one class acquires the properties (methods and fields) of another class to achieve the reusability of code.
- extends is the keyword used to inherit the properties of a class.

Advantages of Inheritance

- Code reusability
- Modifications
- Enhancements
- Software extendibility

Example: The main advantages of inheritance are code reusability and readability.

- In the process of inheritance child class will acquires the properties (methods and fields) of super class.
- So, we can reuse the properties and functionality of parent class in child class also
- No need to write the same code again in child class.
 - This makes it easier to reuse the code
 - Makes us write the less code
 - The code becomes much more readable.

Example:

Vehicle (Tata Company) are a class

Subclasses Can be: Car, Bus, Truck

properties:

- Color
- Price
- Milage
- Model
- Engine

Functionality: Getter setters

Child classes: Male and Female

- Since Car, Bus, Truck are vehicle
- They share some common properties and behaviour's (functionality)
- So, they can inherit those properties and functionality from Vehicle class
- Specific Functionality can be written in their class separately.

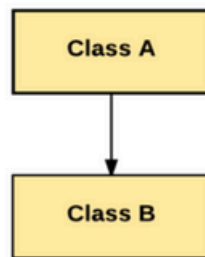
Types of Inheritance:

- Single inheritance
- Multi-level inheritance
- Multiple inheritance
- Hierarchical inheritance
- Hybrid Inheritance

Single inheritance: A Subclass inherit the properties from only one class

Class B extends Class A

In Single Inheritance one class extends another class (one class only).

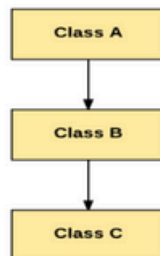


Single Inheritance

In above diagram, Class B extends only Class A. Class A is a super class and Class B is a Sub-class.

Multi-level inheritance: A subclass can be super class of another class

In Multilevel Inheritance, one class can inherit from a derived class. Hence, the derived class becomes the base class for the new class.



Multilevel Inheritance

As per shown in diagram Class C is subclass of B and B is a of subclass Class A.

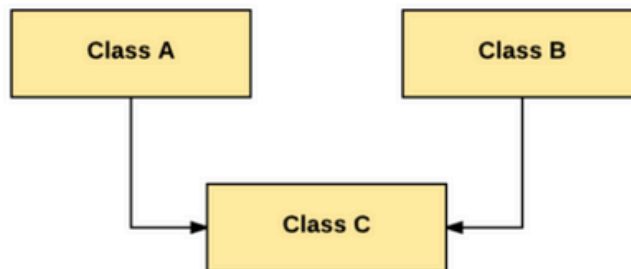
Class B Extends Class A

Class B Extends Class C

Multiple inheritance

- One subclass inherits the properties from multiple super class
- Java doesn't support Multiple inheritance

In Multiple Inheritance, one class extending more than one class. Java does not support multiple inheritance



Multiple Inheritance

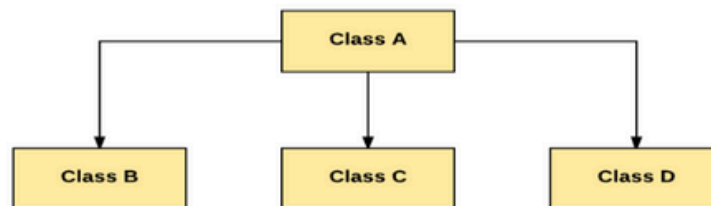
As per above diagram, Class C extends Class A and Class B both.

Class C Extends Class A, B

Hierarchical inheritance

More than One subclass can inherit the properties from one super class

In Hierarchical Inheritance, one class is inherited by many sub classes.



Hierarchical Inheritance

As per above example, Class B, C, and D inherit the same class A.

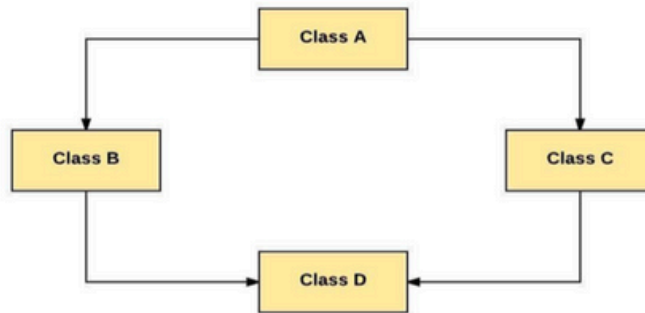
Class B Extends Class A

Class C Extends Class A

Class D Extends Class A

Hybrid Inheritance

- Combination of hierarchical and Multiple inheritance
- It is not supported by java



Hybrid Inheritance

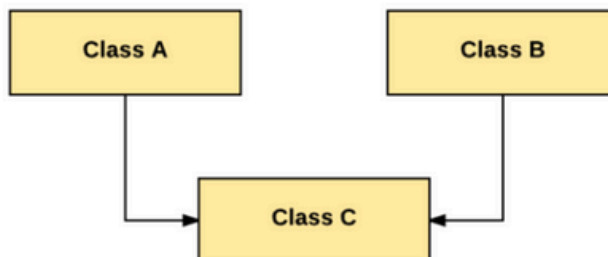
As per above example, all the public and protected members of Class A are inherited into Class D, first via Class B and secondly via Class C.

Note: Java doesn't support hybrid/Multiple inheritance

What is Multiple inheritance in java

- One subclass inherits the properties from multiple super class
- Java doesn't support Multiple inheritance

In Multiple Inheritance, one class extending more than one class. Java does not support multiple inheritance



Multiple Inheritance

As per above diagram, Class C extends Class A and Class B both.

Class C Extends Class A, B is not supported by Java

How to achieve Multiple inheritance in java

- By the using interfaces.
- A class can implement any number of interfaces.
- But it can extend only one class.

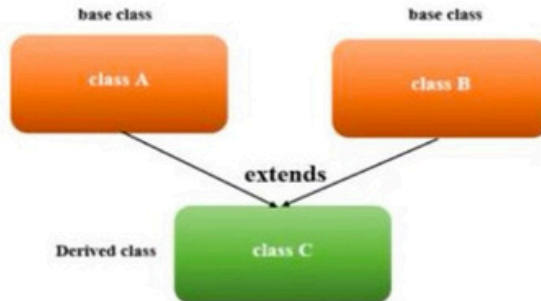
Can multiple inheritance possible in java using abstract class

- No
- Because As per Java rules a class can extend only one class
- whether its normal class or abstract class
- Even abstract class also cause diamond problem

Why multiple inheritance is not possible in java

- Because it can cause to diamond problem in subclass

What is Diamond problem or Problem with multiple inheritance?



- Consider an example
- We have 3 Classes A B C
- class A and B Super class
- C is the sub class inherits the properties from 2 super class A and B
- In Class C we have all features of base classes (class A and class B).
- if both base classes (class A and class B) have the same methods with the same signature.
- It creates ambiguity for the compiler.
- This is called Diamond problem in programming
- That's the main reason why Java doesn't support multiple inheritance.

Task for readers:

These are the most FAQ OOP basics! 🚀

- 📖 **Explore More:** Dive deeper into OOP FAQs, practice, and try small code snippets for each concept 📖. We'll share detailed code snippets on the community channel soon. **For now, we want you to explore and learn!** 🚀
- 🛠️ **For Experienced Professionals:** Reflect on how you've implemented these concepts in your current project.
- 🎓 **For College Students & Graduates:** Relate these concepts to your syllabus and try implementing small projects.

Pro Tip: Think Big, Start Small! 🌟 Let's grow together!

Note:

- This document is meant for brush-ups key concepts & Top FAQ.
- We always recommend working on projects and practicing hands-on coding to solidify your concepts. 📖🚀

Copyright Notice: 📜

🚫 **Personal Use Only:** This content is strictly for personal use.

🚫 **No Unauthorized Sharing:** Do not post on social media (LinkedIn, Instagram, etc.) without permission.

⚖️ **Strict Copyright Action:** If you share this ePDF on your personal Instagram or LinkedIn without permission, your account may be suspended or permanently removed due to copyright strikes ⚠️.

💖 **Share Responsibly:** You can share it privately with friends preparing for interviews, but not on social media 🚫

✉️ **Contact for Permissions:**

- Email us at: support@javatechcommunity.com
- DM us on Instagram: [@java_tech_community](https://www.instagram.com/java_tech_community)